Guía de Actividades **Práctico-Experimentales** Nro. 002

1. Datos Generales

Asignatura	Sistemas Digitales
Ciclo	5to
Unidad	1 – Microcontroladores para sistemas embebidos
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	2
Título de la Práctica	Juego de luces interactivo con ESP32
Nombre del Docente	Luis Darío Sinche C.
Nombre del estudiante	Darwin Ricardo Jimbo Jaramillo
Fecha	23 de octubre de 2026
Horario	Viernes 11h30 a 13h30
Lugar	Laboratorio de computación (simulación en Wokwi y prueba en la placa ESP32 del grupo)
Tiempo planificado en el Sílabo	3 horas (APE, semana 2): 2 h presenciales y 1 h para terminar el informe
Modalidad de trabajo	Cada estudiante simula en su computadora; se trabaja y se entrega en parejas; la prueba física se hace con la placa del grupo
Placa	ESP32 DevKit V1 de 30 pines (módulo ESP32-WROOM-32)

2. Objetivo(s) de la Práctica:

Objetivo. Programar en el ESP32 un juego de luces con cuatro LEDs cuya velocidad cambia con un pulsador, usando temporización no bloqueante con millis() y la lectura del pulsador con pull-up interna, antirrebote y detección de flanco.

Parte A. Circuito y secuencia de luces (Wokwi)

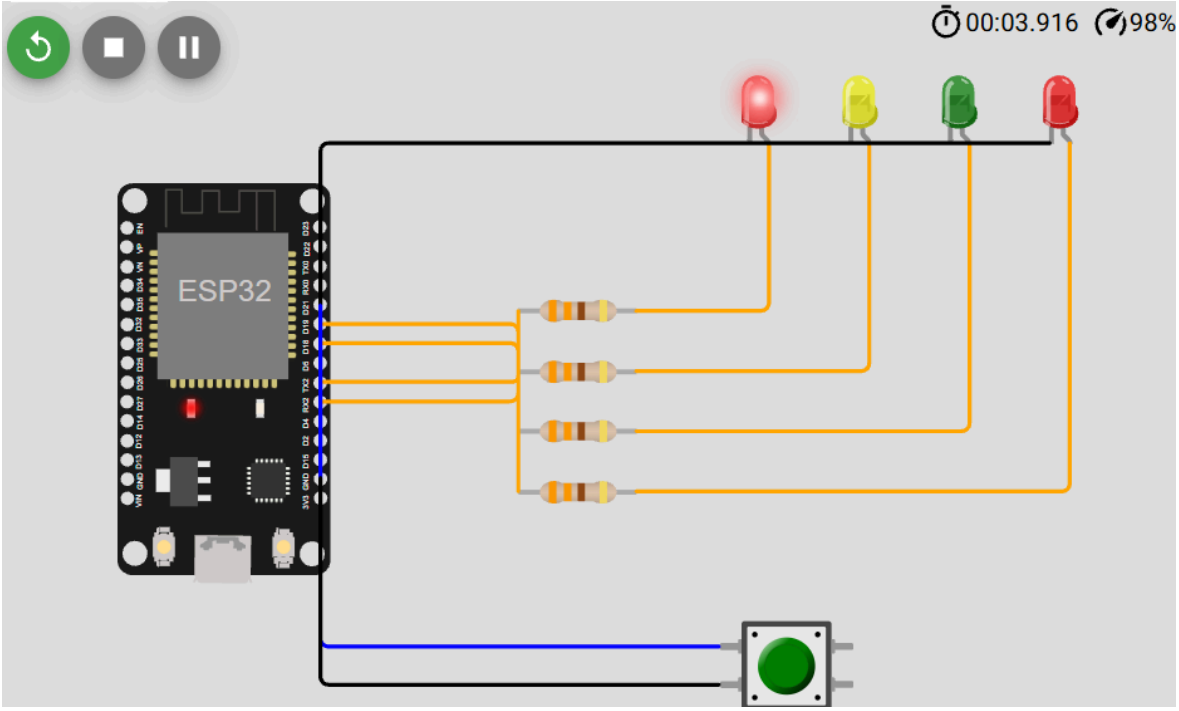
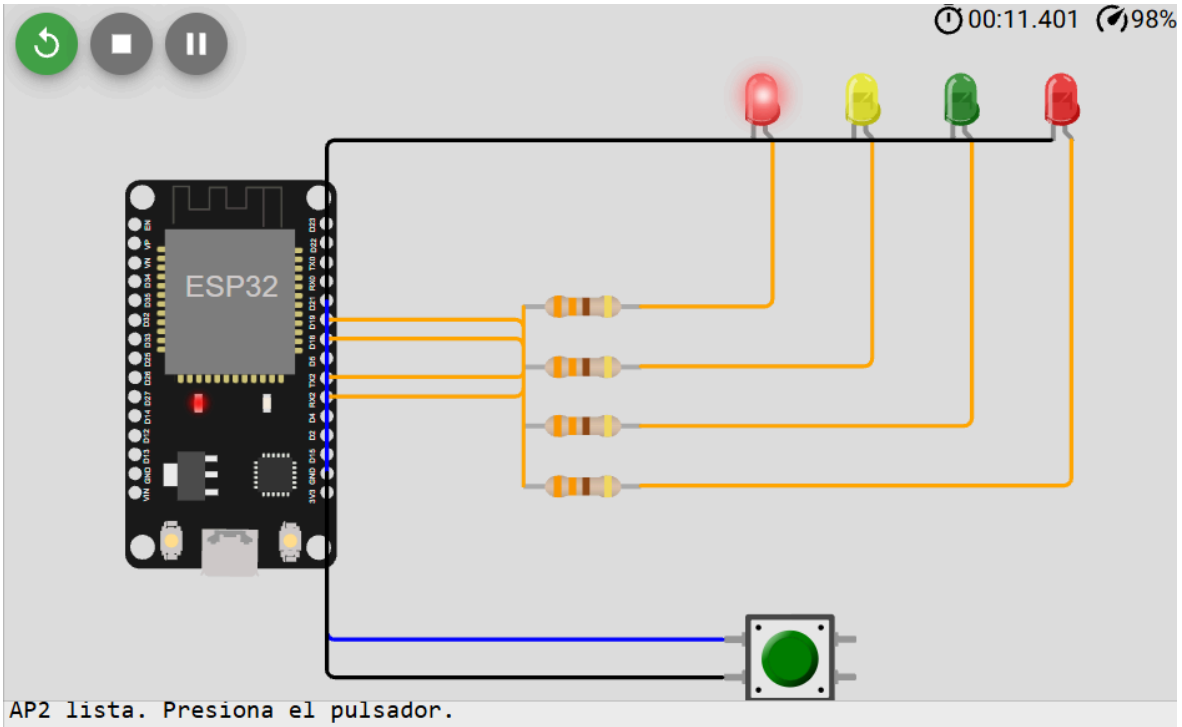


Tabla A. Efecto del periodo

PERIODO (ms)	¿Qué observan?
1000	Las luces led encienden una tras otra de manera más consecutivamente lenta que el valor predeterminado
300	Las luces led encienden una tras otra en el orden presentado según se esperaba
100	Las luces led encienden una tras otra de manera más rápida que las del valor predeterminado

Parte B. El pulsador cambia la velocidad (Wokwi)




```
AP2 lista. Presiona el pulsador.
Periodo: 150 ms
Periodo: 60 ms
Periodo: 400 ms
Periodo: 150 ms
Periodo: 60 ms
Periodo: 400 ms
```

Tabla B. Efecto del antirrebote (10 clics)

T_REBOTE (ms)	Líneas impresas	¿Coincide con los 10 clics?
0	80	No
30	10	Si

Parte C. Prueba en la placa ESP32 real

En esta parte de la actividad, se realizó el armado del circuito de manera manual en el laboratorio de clases, ejecutando correctamente el circuito 2, como evidencia, se grabó un video del funcionamiento del circuito y se lo subió al repositorio: https://github.com/Darwin-J5/AP2_SistemasDigitales_Jimbo_Jaramillo

3. Preguntas de Control:

1. **¿Por qué el Código 1 usa millis() en lugar de delay()? ¿Qué pasaría con el pulsador del Código 2 si cada paso de los LEDs usara delay(400)?**

   El Código 1 usa millis() porque es una función no bloqueante; permite que el microcontrolador siga ejecutando otras tareas mientras evalúa si ya pasó el tiempo necesario. Si el Código 2 usara Delay(400), el procesador se detendría por completo durante 400 milisegundos. Cualquier pulsación del botón que ocurriera durante ese tiempo de pausa sería ignorada, haciendo que el sistema sea poco receptivo o lento a la hora de responder.

2. **El pulsador va entre el GPIO21 y GND, sin resistencia externa. ¿Qué valor lee digitalRead() con el pulsador suelto y con el pulsador presionado? ¿Por qué?**
   Debido a que el pin está configurado como IMPUT_PULLUP. Esta configuración activa una resistencia interna en el ESP32 conectada a 3.3V, lo que mantiene el voltaje en HIGH cuando el circuito está abierto. Al presionar el botón, el pin se conecta directamente a GND, forzando el voltaje a LOW.

3. **¿Qué pasaría si conectaran el pulsador al GPIO34 sin cambiar el código? Justifiquen con lo visto en clase.**

Los pines GPIO34 al GPIO39 en el ESP32 son de solo entrada y no tienen resistencias pull-up ni pull-down internas. Si se conecta el pulsador al GPIO34 sin añadir una resistencia externa, el pin quedaría en estado flotante cuando el botón esté suelto, lo que haría que la velocidad cambiase de forma errática e impredecible sin siquiera tocar el botón.

4. **En el paso B3 la velocidad cambió una sola vez aunque mantuvieron presionado el pulsador. ¿Qué parte del Código 2 lo garantiza?**

   El cambio ocurre una sola vez porque la función botonPresionado() está diseñada para detectar el momento exacto de la transición, no el estado continuo. La validación ocurre en el bloque:

   if ((millis() - tUltimoCambio) >= T_REBOTE && lectura != estadoEstable)

   Solo cuando la lectura ha estado estable durante 30 ms y es diferente al estado guardado anterior, actualiza la variable estadoEstable y retorna true únicamente si ese nuevo estado estable es LOW. En los siguientes ciclos del loop(), como el estado ya no cambia, la condición lectura != estadoEstable es falsa.

5. **Expliquen la Tabla B: ¿por qué con T_REBOTE = 0 puede haber más líneas impresas que clics?**

   Los contactos físicos de un pulsador metálico no se cierran de forma perfecta e instantánea; chocan y rebotan durante unos pocos milisegundos antes de estabilizarse. Si el tiempo de antirrebote es 0, el microcontrolador es tan rápido que lee cada uno de esos pequeños rebotes eléctricos como si fueran pulsaciones distintas. Por eso, un solo clic físico puede registrar múltiples pulsaciones en el código.

6. **Calculen la corriente que circula por un LED rojo (Vf ≈ 2,0 V) con una resistencia de 330 Ω a 3,3 V. ¿Por qué no conviene usar un LED azul?**

   Calculo de la corriente:

   3.3 V - 2.0 V = 1.3 V

   1.3 V / 330 Ohm = 0.00393 A = 3.93 mA

   No conviene usar un LED azul porque estos suelen tener un voltaje de caída más alto, típicamente entre 3.0 V y 3.3 V. Si se alimenta a 3.3 V, apenas sobraría entre 0 V y 0.3 V para la resistencia, resultando en una corriente casi nula.

7. **Mencionen una diferencia entre probar el circuito en Wokwi y probarlo en la placa real.**

   En Wokwi los componentes son ideales, el ruido eléctrico no existe y los contactos de los botones pueden configurarse para no tener rebote mecánico a menos que se simule explícitamente. En la placa real, las conexiones en la protoboard pueden tener falsos contactos, los pines captan ruido electromagnético real, el voltaje no es siempre exactamente 3.3V, y los rebotes del botón varían físicamente dependiendo de cómo se presionen.

8. **El ESP32 sigue funcionando a 240 MHz aunque el juego solo cambie de LED cada 400 ms. Propongan una idea para reducir el consumo de energía si el juego funcionara con batería.**

   Si el juego funciona con batería y la mayor parte del tiempo el microcontrolador solo espera a que pasen 400 ms o a que se presione un botón, se puede reducir la frecuencia de reloj del procesador. Adicionalmente, se podrían usar modos de bajo consumo como el Light Sleep entre los cambios de LED, configurando un temporizador y el pin del botón como fuentes de interrupción para despertar al ESP32 solo cuando sea necesario actuar.

4. Conclusiones

- El uso de millis() en lugar de delay() permite que el microcontrolador ejecute la secuencia de luces sin detenerse, garantizando que el sistema pueda responder al pulsador en todo momento de forma inmediata.
- Para que un botón físico funcione correctamente, no basta con conectarlo. Es indispensable usar configuraciones y algoritmos de software para evitar falsas lecturas provocadas por el ruido electromagnético o el rebote mecánico de los contactos.
- Diseñar este tipo de circuitos requiere entender las características eléctricas de los componentes y reconocer que probar un código en un entorno físico presenta desafíos de ruido y falsos contactos que un simulador idealizado no siempre muestra.

5. Bibliografía

1. Freire Sánchez, L. A. (2023). Electrónica y sistemas embebidos: una visión a nivel técnico y tecnológico. ISBN 978-9942-636-22-5.

2. Zailani, N. F. (2024). Getting started with Arduino and ESP32: A beginner's guide with Wokwi. Politeknik Mersing.

3. Espressif Systems. (s. f.). ESP32 Series Datasheet. https://www.espressif.com/en/support/documents/technical-documents

4. Espressif Systems. (s. f.). Arduino-ESP32 documentation. https://docs.espressif.com/projects/arduino-esp32/

5. CodeMagic LTD. (2025). Wokwi – documentación. https://docs.wokwi.com

6. Arduino. (2024). Arduino Language Reference. https://www.arduino.cc/reference/